

Binary Search Trees (BST)

A Binary Search Tree is a node-based binary tree data structure that maintains elements in a sorted order to allow for fast lookup, addition, and removal. It is a cornerstone of computer science curricula because it demonstrates the trade-off between linear and logarithmic time complexity.

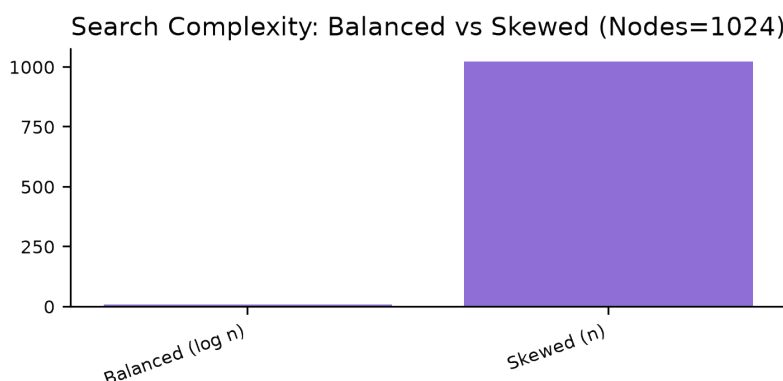
Core Definition and Structural Properties

- Each node contains a key, a left child pointer, and a right child pointer.
- Left Subtree Property: The key of any node in the left subtree is less than the node's key.
- Right Subtree Property: The key of any node in the right subtree is greater than the node's key.
- Recursive Structure: Both the left and right subtrees must also be binary search trees.
- No duplicate keys are allowed in a standard BST implementation.

Mathematical Analysis and Complexity

- The time complexity of search, insert, and delete operations is proportional to the height of the tree.
- In the average case (randomly inserted keys), the tree is relatively balanced, leading to logarithmic performance.
- In the worst case (keys inserted in sorted order), the tree becomes a skewed 'degenerate' tree resembling a linked list.
- The minimum height of a BST with n nodes is $\text{floor}(\log_2(n))$.
- The maximum height of a BST with n nodes is $n-1$.

$$T(n) = \mathcal{O}(h) \text{ where } \log$$

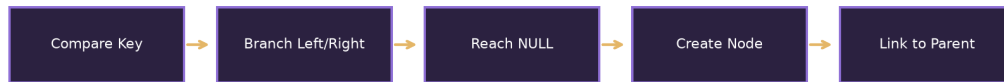


The Search and Insertion Process

- Search: Compare the target key with the current node; move left if smaller, right if larger.
- Search terminates when the key is found or a null pointer is reached.

- Insertion: Always occurs at a leaf position found by following the search logic.
- A new node is never inserted as an internal node; it is always attached to a parent that previously had a null child.

Insertion Algorithm Logic



Deletion Logic and Cases

- Case 1: Node is a leaf. Simply remove the node and set the parent's child pointer to NULL.
- Case 2: Node has one child. Bypass the node by linking its parent directly to its child.
- Case 3: Node has two children. Find the inorder successor (smallest in right subtree) or predecessor (largest in left subtree).
- In Case 3, replace the target node's value with the successor's value, then delete the successor node.
- Successors in Case 3 are guaranteed to have at most one child, simplifying the secondary deletion.

Tree Traversals

- Inorder (Left-Root-Right): Visits nodes in ascending sorted order; most common for BSTs.
- Preorder (Root-Left-Right): Useful for creating a copy of the tree or prefix expressions.
- Postorder (Left-Right-Root): Useful for deleting the tree or postfix expressions.
- All traversals visit each node exactly once, resulting in linear time complexity.

$$T_{traversal}(n) = \Theta(n)$$

Common Applications

- Efficient implementation of 'Set' and 'Map' abstract data types.
- Used in database indexing for fast data retrieval.
- Huffman Coding Trees for data compression algorithms.
- Priority Queues where elements are frequently searched and updated.

Key Terms

Root	The top-most node of the tree from which all other nodes descend.
Leaf	A node with no children, found at the bottom-most level of the tree branches.

Height	The number of edges on the longest path from the root to a leaf.
Inorder Successor	The node with the smallest key strictly greater than the current node's key.
Balanced Tree	A tree where the heights of the left and right subtrees of every node differ by at most one.
Skewed Tree	A tree where every node has only one child, effectively forming a linear list.

Exam Tips

- Always remember that the Inorder Traversal of a BST produces a sorted sequence of keys.
- When drawing a deletion of a node with two children, clearly show the replacement with the Inorder Successor.
- If an exam question asks for the 'worst-case search time', assume a skewed tree and answer $O(n)$.
- Pay attention to whether the question allows duplicate keys; standard BSTs usually do not or handle them via counters.